

Project Part 2: Decentralized Application (DApp) Interface

Presentation Deadline: 21 April 2026

Individual Viva: Understanding of the Project, Smart Contract, DApp, Ethereum blockchain, and Solidity.

Description: Now that your Smart Contract for the Humanitarian Aid & Relief Escrow is ready, you must build the client-side interface using HTML/CSS/JavaScript (or a framework [that you are comfortable with](#) of e.g. React,Vue), communicating via Web3.js/Ethers.js and MetaMask.

Requirements:

- **Note:** Some features are open-ended, so think logically about real-life humanitarian scenarios, what makes sense, and complete the Project accordingly with your own restrictions if needed.

1. Role-Based Dashboard

- The UI must detect the active MetaMask account and display the correct view of a [Donor](#), [Relief_Agency](#), or [UN_Arbiter](#).
- **Registration:** A form for new users to register their Name and Role. The wallet address should be pulled automatically from MetaMask.
- **Validation:** Prevent submitting the [registration form](#) if the fields are empty or if the wallet is already registered.
Ensure the UI reflects the default Reputation Score of 100 for newly registered Relief Agencies.

2. Global Relief Mission Feed (Filtering & Sorting)

- **View:** Display a list of all [Pending](#) missions available for funding.
- **Sorting:** Sort missions by **Highest Max Budget first** to prioritize critical aid.
- **Filtering:** Add a dropdown to filter missions by Category (e.g., Show only "Medical" missions) or by Region.
- **Note:** Perform this sorting/filtering in an **efficient manner** on the frontend.

3. Donor Interaction (Posting & Funding)

- **Note:** The following features are **restricted to Donor**.
- **Post Mission Form:** Inputs for Category, Region, and Max Budget (in Ether).
- **View Pledges:** When a Donor clicks on a specific mission they created, show a list of received operational cost pledges from Relief Agencies.
- **"Fund Mission" Action:** A **"Fund"** button next to each pledge.

- **Payment Trigger:** Clicking **"Fund"** must trigger a MetaMask transaction requiring the user to pay the exact Bid Amount. (If your contract allows sending excess Ether for a refund, ensure the UI explains this to the user).
- **Success:** Show a distinct "Funds in Escrow" badge after the transaction confirms.
- **Manage Delivery:** Provide buttons for the Donor to **"Approve Delivery"** or **"Dispute"** the mission once an agency marks it as delivered.

4. Relief Agency Interaction (Pledging)

- **Pledge Form:** Only visible to **Relief_Agency** accounts on **Pending** missions.
- **Validation:** Prevent the **"Submit Pledge"** button from working if the input operational cost is higher than the Mission's Max Budget.
- **Reputation Lock:** Disable the pledge button entirely (or show an error message) if the connected agency's Reputation Score is **below 40**.
- **My Missions:** A private section showing missions where the **Relief_Agency** has been successfully funded (**In_Transit**). Include a **"Mark as Delivered"** button.

5. UN_Arbiter Panel

- A private section visible only to the **UN_Arbiter**.
- Shows a list of missions with the status **Disputed**.
- Two action buttons for resolution: "Outcome A (**Refund Donor / Penalize Agency**)" and "Outcome B (**Pay Agency / False Alarm**)". **[Follow Smart Contract]**
- **Treasury Display:** Display the **Total Operational Fees** (platform tax) collected by the contract (Contract Balance).

Technical Constraints:

- **No Database:** You are not allowed to use MongoDB, SQL, or any off-chain database. All mission details, pledges, user profiles, and reputation scores must be fetched directly from the Ethereum Blockchain.
- **Auto-Refresh:** The UI should update automatically (e.g., using Events) when a pledge is placed or a status changes, without requiring a full page reload.
- **Tools:** Use **Ganache** for the local blockchain and **Truffle** for deployment.